

Generated from Markdown

Spring AI Alibaba Agent 傻瓜式落地手册

0. 这份文档解决什么问题

你现在的目标不是一次性掌握所有 Agent 技术，而是在当前 Spring Boot 项目里先跑通一个可演示、可写进简历的 AI Agent 闭环。

第一版目标：

```
graph TD
    A[输入采购订单号] --> B[Agent 调用业务 Tool]
    B --> C[查询采购订单、到货、入库数据]
    C --> D[Java 规则判断阻塞阶段]
    D --> E[大模型生成解释和建议]
    E --> F[接口返回诊断结果]
```

第一版先做：

流程阻塞诊断 Agent

跑通后再扩展：

采购执行预警 Agent
供应商履约评分 Agent

本文档会给你一套可以照着写的流程，包括：

- Maven 依赖怎么加
- 配置怎么写
- 包结构怎么建
- Mapper 怎么查业务快照
- Tool 怎么封装
- ReactAgent 怎么创建
- Controller / Service 怎么写
- Apifox 怎么测
- 常见报错怎么处理

参考官方文档：

- [Spring AI Alibaba 版本说明](#)
- [Spring AI Alibaba Quick Start](#)
- [Spring AI Alibaba Agents](#)
- [Spring AI Alibaba Tools](#)
- [Spring AI Alibaba Memory](#)

1. 第一版总体路线

第一版不要上来就做三个 Agent，也不要上来就做 Graph / Workflow。

先按这个顺序：

1. 加 Maven 依赖
2. 配置 DashScope API Key
3. 写一个普通 chat 接口，验证模型能通
4. 写一个 AgentQueryMapper，查询订单执行快照
5. 写一个 PurchaseOrderSnapshotTool，给 Agent 调用
6. 写 ReactAgent 配置
7. 写 ProcessDiagnosisAgentService
8. 写 AgentController
9. Apifox 输入 orderNo 测试

跑通后，你会得到一个接口：

```
POST /agent/diagnose/order
```

请求：

```
{
  "orderNo": "P02026040011",
  "threadId": "u1-order-diagnose"
}
```

返回类似：

```
{
  "code": 200,
  "msg": "success",
  "data": {
    "orderNo": "P02026040011",
    "currentStage": "入库确认阶段",
    "blockReason": "订单已全部到货，但仍有部分数量未完成入库。",
    "suggestOwner": "WAREHOUSE",
    "suggestAction": "请仓库岗检查待确认入库单并执行确认入库。",
    "aiSummary": "P02026040011 当前卡在入库确认阶段..."
  }
}
```

2. 修改 pom.xml

2.1 你现在为什么依赖标红

如果你写了：

```
<dependency>
  <groupId>com.alibaba.cloud.ai</groupId>
  <artifactId>spring-ai-alibaba-agent-framework</artifactId>
</dependency>
```

但是没有写 `version`，也没有 BOM，那么 Maven 会直接报：

```
'dependencies.dependency.version' is missing
```

解决方式有两个。

2.2 推荐方式：使用 BOM

在 `pom.xml` 的 `<properties>` 后面、`<dependencies>` 前面加入：

```

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.alibaba.cloud.ai</groupId>
      <artifactId>spring-ai-alibaba-bom</artifactId>
      <version>1.1.2.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>

    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>1.1.2</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>

    <dependency>
      <groupId>com.alibaba.cloud.ai</groupId>
      <artifactId>spring-ai-alibaba-extensions-bom</artifactId>
      <version>1.1.2.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

然后在 `<dependencies>` 里加入：

```

<!-- Spring AI Alibaba Agent Framework -->
<dependency>
  <groupId>com.alibaba.cloud.ai</groupId>
  <artifactId>spring-ai-alibaba-agent-framework</artifactId>
</dependency>

<!-- DashScope 模型接入 -->
<dependency>
  <groupId>com.alibaba.cloud.ai</groupId>
  <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
</dependency>

```

2.3 简单方式：直接写版本

如果你只是想先跑起来，也可以直接写版本：

```
<dependency>
  <groupId>com.alibaba.cloud.ai</groupId>
  <artifactId>spring-ai-alibaba-agent-framework</artifactId>
  <version>1.1.2.0</version>
</dependency>

<dependency>
  <groupId>com.alibaba.cloud.ai</groupId>
  <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
  <version>1.1.2.0</version>
</dependency>
```

注意：

- 官方版本页推荐 SAA `1.1.2.0` 对应 Spring Boot `3.5.x`。
- 你的项目当前是 Spring Boot `3.5.12`，方向是匹配的。
- 如果 `spring-ai-alibaba-starter-dashscope:1.1.2.1` 拉不下来，先用 `1.1.2.0`。

2.4 刷新 Maven

修改后执行：

```
cd D:\code\project\inventory\inventory_back
mvn -q -DskipTests compile
```

如果 IDEA 还红：

Maven 面板 -> Reload All Maven Projects

3. 配置 application-dev.yml

在 `inventory_back/src/main/resources/application-dev.yml` 加：

```
spring:
  ai:
    dashscope:
      api-key: ${AI_DASHSCOPE_API_KEY}
```

你的文件里已经有 `spring.datasource`，所以注意合并成一个 `spring:`，不要写两个顶级 `spring:` 覆盖掉。

示例结构：

```
spring:
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://xxx
    username: root
    password: xxx
  ai:
    dashscope:
      api-key: ${AI_DASHSCOPE_API_KEY}
```

本地设置环境变量：

```
$env:AI_DASHSCOPE_API_KEY="你的百炼 API Key"
```

如果用 IDEA 启动，也可以在 Run Configuration 里加环境变量：

```
AI_DASHSCOPE_API_KEY=你的百炼 API Key
```

4. 新增包结构

在 `inventory_back/src/main/java/com/xixi` 下新增：

```
agent
├── config
├── controller
├── dto
├── mapper
├── service
├── tool
└── vo
```

完整路径：

```
com.xixi.agent.config
com.xixi.agent.controller
com.xixi.agent.dto
com.xixi.agent.mapper
com.xixi.agent.service
com.xixi.agent.tool
com.xixi.agent.vo
```

5. 第一步：先写一个普通 Agent Chat 接口

这一步只验证模型能不能通，不接业务数据。

5.1 新建 AgentChatRequest

文件:

```
inventory_back/src/main/java/com/xixi/agent/dto/AgentChatRequest.java
```

代码:

```
package com.xixi.agent.dto;

import lombok.Data;

@Data
public class AgentChatRequest {
    private String message;

    private String threadId;
}
```

5.2 新建 AgentConfig

文件:

```
inventory_back/src/main/java/com/xixi/agent/config/AgentConfig.java
```

代码:

```
package com.xixi.agent.config;

import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.alibaba.cloud.ai.graph.checkpoint.savers.MemorySaver;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AgentConfig {

    @Bean
    public ReactAgent inventoryChatAgent(ChatModel chatModel) {
        return ReactAgent.builder()
            .name("inventory_chat_agent")
            .model(chatModel)
            .systemPrompt("""
                你是供应商协同采购入库系统的智能助手。
                当前阶段只允许回答系统相关问题，不允许编造业务数据。
                如果用户询问具体订单、库存、供应商信息，但你没有工具数据，请说明需要接入业务
                工具。
            """)
            .saver(new MemorySaver())
            .build();
    }
}
```

如果 `MemorySaver` 导包失败，确认包名：

```
import com.alibaba.cloud.ai.graph.checkpoint.savers.MemorySaver;
```

如果 `ReactAgent` 导包失败，确认包名：

```
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
```

5.3 新建 AgentController

文件：

```
inventory_back/src/main/java/com/xixi/agent/controller/AgentController.java
```

代码：

```

package com.xixi.agent.controller;

import com.alibaba.cloud.ai.graph RunnableConfig;
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.xixi.agent.dto.AgentChatRequest;
import com.xixi.pojo.vo.Result;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/agent")
@RequiredArgsConstructor
public class AgentController {
    @Qualifier("inventoryChatAgent")
    private final ReactAgent inventoryChatAgent;

    @PostMapping("/chat")
    public Result chat(@RequestBody AgentChatRequest request) {
        String threadId = request.getThreadId() == null ||
request.getThreadId().isBlank()
            ? "default-thread"
            : request.getThreadId();

        RunnableConfig config = RunnableConfig.builder()
            .threadId(threadId)
            .build();

        AssistantMessage response = inventoryChatAgent.call(request.getMessage(),
config);
        return Result.success(response.getText());
    }
}

```

5.4 放行测试接口

如果你当前 Spring Security 默认拦截所有接口，为了 Apifox 测试，可以临时在 `SpringSecurityConfig` 中放行：

```

.requestMatchers(
    "/auth/login",
    "/error",
    "/agent/**"
).permitAll()

```

5.5 Apifox 测试

接口:

```
POST http://localhost:8080/agent/chat
```

Body:

```
{
  "message": "你好, 你是什么助手? ",
  "threadId": "test-001"
}
```

如果模型正常, 会返回一段文本。

如果这一步都不通, 不要继续写业务 Agent, 先排查依赖、API Key、模型配置。

6. 第二步：做订单执行快照查询

这一步不接大模型, 先用 SQL 查出订单执行快照。

6.1 新建 OrderSnapshotVO

文件:

```
inventory_back/src/main/java/com/xixi/agent/vo/OrderSnapshotVO.java
```

代码:

```
package com.xixi.agent.vo;

import lombok.Data;

import java.math.BigDecimal;

@Data
public class OrderSnapshotVO {
    private Long orderId;

    private String orderNo;

    private String status;

    private Long supplierId;

    private String supplierName;

    private BigDecimal totalOrderNumber;

    private BigDecimal totalArrivedNumber;

    private BigDecimal totalInboundNumber;

    private Integer arrivalCount;

    private Integer inboundCount;
}
```

6.2 新建 AgentQueryMapper

文件:

```
inventory_back/src/main/java/com/xixi/agent/mapper/AgentQueryMapper.java
```

代码:

```
package com.xixi.agent.mapper;

import com.xixi.agent.vo.OrderSnapshotVO;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;

@Mapper
public interface AgentQueryMapper {
    OrderSnapshotVO getOrderSnapshotByOrderNo(@Param("orderNo") String orderNo);
}
```

6.3 新建 AgentQueryMapper.xml

文件:

```
inventory_back/src/main/resources/com/xixi/mapper/AgentQueryMapper.xml
```

代码:

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.xixi.agent.mapper.AgentQueryMapper">
    <select id="getOrderSnapshotByOrderNo"
        resultType="com.xixi.agent.vo.OrderSnapshotVO">
        select po.id as orderId,
            po.order_no as orderNo,
            po.status as status,
            po.supplier_id as supplierId,
            s.name as supplierName,
            coalesce(sum(poi.order_number), 0) as totalOrderNumber,
            coalesce(sum(poi.arrived_number), 0) as totalArrivedNumber,
            coalesce(sum(poi.inbound_number), 0) as totalInboundNumber,
            (
                select count(1)
                from arrival a
                where a.order_id = po.id
                    and a.deleted = 0
            ) as arrivalCount,
            (
                select count(1)
                from inbound i
                left join arrival a on i.arrival_id = a.id
                where a.order_id = po.id
                    and i.deleted = 0
                    and a.deleted = 0
            ) as inboundCount
        from purchase_order po
        left join supplier s on po.supplier_id = s.id
        left join purchase_order_item poi on po.id = poi.order_id and poi.deleted = 0
        where po.order_no = #{orderNo}
            and po.deleted = 0
        group by po.id, po.order_no, po.status, po.supplier_id, s.name
    </select>
</mapper>
```

6.4 先编译

```
mvn -q -DskipTests compile
```

如果 Mapper XML 路径正确，你当前配置：

```
mybatis-plus:
  mapper-locations: classpath*:com/xixi/mapper/*.xml
```

可以扫描到这个 XML。

7. 第三步：写 Tool 输入对象

7.1 新建 PurchaseOrderSnapshotRequest

文件：

```
inventory_back/src/main/java/com/xixi/agent/dto/PurchaseOrderSnapshotRequest.java
```

代码：

```
package com.xixi.agent.dto;

import lombok.Data;

@Data
public class PurchaseOrderSnapshotRequest {
    private String orderNo;
}
```

8. 第四步：写 PurchaseOrderSnapshotTool

这个 Tool 给 Agent 调用。

8.1 新建 PurchaseOrderSnapshotTool

文件：

```
inventory_back/src/main/java/com/xixi/agent/tool/PurchaseOrderSnapshotTool.java
```

代码：

```

package com.xixi.agent.tool;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.xixi.agent.dto.PurchaseOrderSnapshotRequest;
import com.xixi.agent.mapper.AgentQueryMapper;
import com.xixi.agent.vo.OrderSnapshotVO;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.model.ToolContext;
import org.springframework.stereotype.Component;

import java.util.Map;
import java.util.function.BiFunction;

@Component
@RequiredArgsConstructor
public class PurchaseOrderSnapshotTool implements
BiFunction<PurchaseOrderSnapshotRequest, ToolContext, String> {
    private final AgentQueryMapper agentQueryMapper;
    private final ObjectMapper objectMapper;

    @Override
    public String apply(PurchaseOrderSnapshotRequest request, ToolContext toolContext) {
        try {
            if (request == null || request.getOrderNo() == null ||
request.getOrderNo().trim().isEmpty()) {
                return objectMapper.writeValueAsString(Map.of(
                    "success", false,
                    "message", "orderNo不能为空"
                ));
            }

            OrderSnapshotVO snapshot =
agentQueryMapper.getOrderSnapshotByOrderNo(request.getOrderNo().trim());
            if (snapshot == null) {
                return objectMapper.writeValueAsString(Map.of(
                    "success", false,
                    "message", "采购订单不存在",
                    "orderNo", request.getOrderNo()
                ));
            }

            return objectMapper.writeValueAsString(Map.of(
                "success", true,
                "data", snapshot
            ));
        } catch (Exception e) {
            return "{\"success\":false,\"message\":\"工具执行失败\"}";
        }
    }
}

```

注意:

- Tool 返回 JSON 字符串，方便大模型理解。
- Tool 只查询，不修改数据库。
- Tool 内部调用 Mapper，不让模型直接写 SQL。

9. 第五步：配置带 Tool 的流程诊断 Agent

修改刚才的 `AgentConfig`。

文件：

```
inventory_back/src/main/java/com/xixi/agent/config/AgentConfig.java
```

建议改成：

```

package com.xixi.agent.config;

import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.alibaba.cloud.ai.graph.agent.hook.modelcalllimit.ModelCallLimitHook;
import com.alibaba.cloud.ai.graph.checkpoint.savers.MemorySaver;
import com.xixi.agent.dto.PurchaseOrderSnapshotRequest;
import com.xixi.agent.tool.PurchaseOrderSnapshotTool;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.tool.ToolCallback;
import org.springframework.ai.tool.function.FunctionToolCallback;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AgentConfig {

    @Bean
    public ReactAgent inventoryChatAgent(ChatModel chatModel) {
        return ReactAgent.builder()
            .name("inventory_chat_agent")
            .model(chatModel)
            .systemPrompt("""
                你是供应商协同采购入库系统的智能助手。
                当前阶段只允许回答系统相关问题，不允许编造业务数据。
            """)
            .saver(new MemorySaver())
            .build();
    }

    @Bean
    public ReactAgent processDiagnosisAgent(ChatModel chatModel,
                                             PurchaseOrderSnapshotTool
purchaseOrderSnapshotTool) {
        ToolCallback orderSnapshotTool = FunctionToolCallback
            .builder("getPurchaseOrderSnapshot", purchaseOrderSnapshotTool)
            .description("根据采购订单号查询采购订单执行快照，包括订单状态、采购总数量、已到货
数量、已入库数量、到货次数、入库次数")
            .inputType(PurchaseOrderSnapshotRequest.class)
            .build();

        ModelCallLimitHook limitHook = ModelCallLimitHook.builder()
            .runLimit(5)
            .exitBehavior(ModelCallLimitHook.ExitBehavior.ERROR)
            .build();

        return ReactAgent.builder()
            .name("process_diagnosis_agent")
            .model(chatModel)
            .systemPrompt("""
                你是供应商协同采购入库系统的流程阻塞诊断助手。
                你必须先调用工具 getPurchaseOrderSnapshot 查询订单执行快照，再进行回答。
                你只能基于工具返回的数据进行分析，不允许编造订单、数量、供应商、仓库信息。
            """)
    }
}

```

理。

你的任务是解释采购订单当前卡在哪个环节、为什么卡住、下一步应该由哪个角色处理。

输出内容必须包含：当前阶段、阻塞原因、证据、建议处理角色、建议动作。
如果工具返回采购订单不存在，请直接说明订单不存在。

""")

```
.tools(orderSnapshotTool)
.hooks(limitHook)
.saver(new MemorySaver())
.build();
```

```
}
```

```
}
```

10. 第六步：写诊断请求和返回对象

10.1 OrderDiagnosisRequest

文件：

```
inventory_back/src/main/java/com/xixi/agent/dto/OrderDiagnosisRequest.java
```

代码：

```
package com.xixi.agent.dto;

import lombok.Data;

@Data
public class OrderDiagnosisRequest {
    private String orderNo;

    private String threadId;
}
```

10.2 OrderDiagnosisVO

文件：

```
inventory_back/src/main/java/com/xixi/agent/vo/OrderDiagnosisVO.java
```

代码：

```
package com.xixi.agent.vo;

import lombok.Data;

import java.util.List;

@Data
public class OrderDiagnosisVO {
    private String orderNo;

    private String currentStage;

    private String blockReason;

    private List<String> evidence;

    private String suggestOwner;

    private String suggestAction;

    private String aiSummary;
}
```

11. 第七步：写 Java 规则判断工具类

这一步很重要。

不要把所有判断都交给大模型。

订单阻塞阶段应该由 Java 代码按状态机判断。

11.1 新建 OrderDiagnosisRule

文件：

```
inventory_back/src/main/java/com/xixi/agent/service/OrderDiagnosisRule.java
```

代码：

```

package com.xixi.agent.service;

import com.xixi.agent.vo.OrderDiagnosisVO;
import com.xixi.agent.vo.OrderSnapshotVO;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

public class OrderDiagnosisRule {

    public static OrderDiagnosisVO diagnose(OrderSnapshotVO snapshot) {
        OrderDiagnosisVO vo = new OrderDiagnosisVO();
        vo.setOrderNo(snapshot.getOrderNo());

        BigDecimal totalOrder = safe(snapshot.getTotalOrderNumber());
        BigDecimal totalArrived = safe(snapshot.getTotalArrivedNumber());
        BigDecimal totalInbound = safe(snapshot.getTotalInboundNumber());

        List<String> evidence = new ArrayList<>();
        evidence.add("订单状态为 " + snapshot.getStatus());
        evidence.add("采购总数量 " + totalOrder);
        evidence.add("已到货数量 " + totalArrived);
        evidence.add("已入库数量 " + totalInbound);
        evidence.add("到货次数 " + snapshot.getArrivalCount());
        evidence.add("入库次数 " + snapshot.getInboundCount());
        vo.setEvidence(evidence);

        if ("WAIT_CONFIRM".equals(snapshot.getStatus())) {
            vo.setCurrentStage("供应商确认阶段");
            vo.setBlockReason("采购订单仍处于待供应商确认状态。");
            vo.setSuggestOwner("SUPPLIER");
            vo.setSuggestAction("请供应商确认订单并反馈预计交期。");
            return vo;
        }

        if ("IN_PROGRESS".equals(snapshot.getStatus()) &&
totalArrived.compareTo(BigDecimal.ZERO) == 0) {
            vo.setCurrentStage("供应商发货 / 仓库到货登记阶段");
            vo.setBlockReason("订单已进入执行中，但目前还没有到货记录。");
            vo.setSuggestOwner("SUPPLIER / WAREHOUSE");
            vo.setSuggestAction("请采购员跟进供应商发货，仓库岗收到货后登记到货。");
            return vo;
        }

        if ("PARTIAL_ARRIVAL".equals(snapshot.getStatus()) &&
totalArrived.compareTo(totalOrder) < 0) {
            vo.setCurrentStage("剩余到货阶段");
            vo.setBlockReason("订单已有部分到货，但仍有剩余采购数量未到货。");
            vo.setSuggestOwner("SUPPLIER");
            vo.setSuggestAction("请采购员催促供应商补齐剩余到货。");
            return vo;
        }
    }
}

```

```

    }

    if ("PARTIAL_ARRIVAL".equals(snapshot.getStatus()))
        && totalArrived.compareTo(totalOrder) >= 0
        && totalInbound.compareTo(totalOrder) < 0) {
        vo.setCurrentStage("入库确认阶段");
        vo.setBlockReason("订单已全部到货, 但仍有部分数量未确认入库。");
        vo.setSuggestOwner("WAREHOUSE");
        vo.setSuggestAction("请仓库岗检查待确认入库单并执行确认入库。");
        return vo;
    }

    if ("COMPLETED".equals(snapshot.getStatus())) {
        vo.setCurrentStage("流程已完成");
        vo.setBlockReason("采购订单已完成, 无阻塞。");
        vo.setSuggestOwner("NONE");
        vo.setSuggestAction("无需处理。");
        return vo;
    }

    if ("CLOSED".equals(snapshot.getStatus()) ||
"CANCELLED".equals(snapshot.getStatus())) {
        vo.setCurrentStage("流程已终止");
        vo.setBlockReason("采购订单已关闭或取消。");
        vo.setSuggestOwner("PURCHASER");
        vo.setSuggestAction("如需继续采购, 请重新发起采购申请或创建新订单。");
        return vo;
    }

    vo.setCurrentStage("未知阶段");
    vo.setBlockReason("当前状态无法根据规则判断阻塞点。");
    vo.setSuggestOwner("PURCHASER");
    vo.setSuggestAction("请采购员人工检查订单状态和明细数据。");
    return vo;
}

private static BigDecimal safe(BigDecimal value) {
    return value == null ? BigDecimal.ZERO : value;
}
}

```

12. 第八步：写 ProcessDiagnosisAgentService

12.1 新建 Service

文件：

```
inventory_back/src/main/java/com/xixi/agent/service/ProcessDiagnosisAgentService.java
```

代码：

```

package com.xixi.agent.service;

import com.alibaba.cloud.ai.graph RunnableConfig;
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.xixi.agent.dto.OrderDiagnosisRequest;
import com.xixi.agent.mapper.AgentQueryMapper;
import com.xixi.agent.vo.OrderDiagnosisVO;
import com.xixi.agent.vo.OrderSnapshotVO;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class ProcessDiagnosisAgentService {
    private final AgentQueryMapper agentQueryMapper;

    @Qualifier("processDiagnosisAgent")
    private final ReactAgent processDiagnosisAgent;

    public OrderDiagnosisVO diagnose(OrderDiagnosisRequest request) {
        if (request.getOrderNo() == null || request.getOrderNo().trim().isEmpty()) {
            OrderDiagnosisVO vo = new OrderDiagnosisVO();
            vo.setBlockReason("订单号不能为空");
            vo.setSuggestAction("请输入采购订单号, 例如 P02026040011。");
            return vo;
        }

        String orderNo = request.getOrderNo().trim();
        OrderSnapshotVO snapshot = agentQueryMapper.getOrderSnapshotByOrderNo(orderNo);
        if (snapshot == null) {
            OrderDiagnosisVO vo = new OrderDiagnosisVO();
            vo.setOrderNo(orderNo);
            vo.setCurrentStage("无法诊断");
            vo.setBlockReason("采购订单不存在");
            vo.setSuggestOwner("PURCHASER");
            vo.setSuggestAction("请确认订单号是否正确。");
            return vo;
        }

        OrderDiagnosisVO ruleResult = OrderDiagnosisRule.diagnose(snapshot);

        String threadId = request.getThreadId() == null ||
            request.getThreadId().isBlank()
            ? "diagnose-" + orderNo
            : request.getThreadId();

        RunnableConfig config = RunnableConfig.builder()
            .threadId(threadId)
            .build();
    }

```

```

String prompt = """
    请诊断采购订单 %s 的流程阻塞情况。
    Java 规则已经给出初步判断：
    当前阶段：%s
    阻塞原因：%s
    建议处理角色：%s
    建议动作：%s

    请你调用工具 getPurchaseOrderSnapshot 核对订单快照，然后基于事实生成一段简洁的中
    文解释。

    不要编造工具中没有返回的数据。
    """.formatted(
        orderNo,
        ruleResult.getCurrentStage(),
        ruleResult.getBlockReason(),
        ruleResult.getSuggestOwner(),
        ruleResult.getSuggestAction()
    );

    AssistantMessage response = processDiagnosisAgent.call(prompt, config);
    ruleResult.setAiSummary(response.getText());
    return ruleResult;
}
}

```

说明：

- `AgentQueryMapper` 是 Java 规则用的。
- `processDiagnosisAgent` 是 LLM 总结用的。
- 即使模型不稳定，Java 规则结果也能返回。

13. 第九步：补 AgentController 诊断接口

修改：

```
inventory_back/src/main/java/com/xixi/agent/controller/AgentController.java
```

加入 `ProcessDiagnosisAgentService`：

```

package com.xixi.agent.controller;

import com.alibaba.cloud.ai.graph RunnableConfig;
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.xixi.agent.dto.AgentChatRequest;
import com.xixi.agent.dto.OrderDiagnosisRequest;
import com.xixi.agent.service.ProcessDiagnosisAgentService;
import com.xixi.pojo.vo.Result;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/agent")
@RequiredArgsConstructor
public class AgentController {
    @Qualifier("inventoryChatAgent")
    private final ReactAgent inventoryChatAgent;

    private final ProcessDiagnosisAgentService processDiagnosisAgentService;

    @PostMapping("/chat")
    public Result chat(@RequestBody AgentChatRequest request) {
        String threadId = request.getThreadId() == null ||
request.getThreadId().isBlank()
            ? "default-thread"
            : request.getThreadId();

        RunnableConfig config = RunnableConfig.builder()
            .threadId(threadId)
            .build();

        AssistantMessage response = inventoryChatAgent.call(request.getMessage(),
config);
        return Result.success(response.getText());
    }

    @PostMapping("/diagnose/order")
    public Result diagnoseOrder(@RequestBody OrderDiagnosisRequest request) {
        return Result.success(processDiagnosisAgentService.diagnose(request));
    }
}

```

14. 第十步：编译和启动

执行：

```
cd D:\code\project\inventory\inventory_back
mvn -q -DskipTests compile
```

再启动项目。

如果启动失败，先看错误类型：

- 依赖找不到：看第 2 节。
- ChatModel 注入失败：看第 17.2 节。
- API Key 缺失：看第 3 节。
- Mapper XML 找不到：确认 XML 放在 `src/main/resources/com/xixi/mapper`。

15. 第十一步：Apifox 测试

15.1 测普通聊天

接口：

```
POST http://localhost:8080/agent/chat
```

Body：

```
{
  "message": "你好，你是什么助手？",
  "threadId": "test-chat-001"
}
```

预期：

```
{
  "code": 200,
  "msg": "success",
  "data": "...
}
```

15.2 测订单诊断

接口：

```
POST http://localhost:8080/agent/diagnose/order
```

Body：

```
{
  "orderNo": "P02026040011",
  "threadId": "diagnose-001"
}
```

预期:

```
{
  "code": 200,
  "msg": "success",
  "data": {
    "orderNo": "P02026040011",
    "currentStage": "...",
    "blockReason": "...",
    "evidence": [
      "订单状态为 ...",
      "采购总数量 ...",
      "已到货数量 ...",
      "已入库数量 ..."
    ],
    "suggestOwner": "...",
    "suggestAction": "...",
    "aiSummary": "..."
  }
}
```

15.3 测不存在订单

Body:

```
{
  "orderNo": "PO_NOT_EXISTS",
  "threadId": "diagnose-002"
}
```

预期:

```
{
  "code": 200,
  "msg": "success",
  "data": {
    "orderNo": "PO_NOT_EXISTS",
    "currentStage": "无法诊断",
    "blockReason": "采购订单不存在",
    "suggestOwner": "PURCHASER",
    "suggestAction": "请确认订单号是否正确。"
  }
}
```

16. 第二个 Agent：采购执行预警 Agent 详细落地步骤

跑通流程阻塞诊断后，第二个最适合做的是采购执行预警 Agent。

这个 Agent 和第一个 Agent 的最大区别是：

第一个 Agent：诊断单个订单
第二个 Agent：扫描一批订单并生成风险列表

16.1 先理解这个 Agent 的职责

它不是“用户问一句，模型答一句”的聊天型 Agent，而是：

系统扫描业务数据
↓
Java 规则先找出风险点
↓
LLM 生成汇总和优先级建议

注意：

- 风险规则用 Java 写，不要完全交给模型判断。
- 模型只负责写摘要、排序建议、解释文本。
- 第一版只做只读分析，不做自动催单、自动改状态。

16.2 第一版建议只做这 5 条预警规则

1. WAIT_CONFIRM 超过 1 天
2. IN_PROGRESS 超过 3 天无到货
3. PARTIAL_ARRIVAL 超过 3 天无新增到货
4. 到货单已创建但未生成入库单
5. 入库单 PENDING 超过 1 天

如果你后面补了库存台账查询，再加：

6. 库存低于安全库存

16.3 推荐包结构

建议新增这些类：

```
inventory_back/src/main/java/com/xixi/agent/dto/WarningScanRequest.java
inventory_back/src/main/java/com/xixi/agent/vo/WarningItemVO.java
inventory_back/src/main/java/com/xixi/agent/vo/WarningScanVO.java
inventory_back/src/main/java/com/xixi/agent/mapper/AgentWarningMapper.java
inventory_back/src/main/resources/com/xixi/mapper/AgentWarningMapper.xml
inventory_back/src/main/java/com/xixi/agent/tool/ProcurementWarningSnapshotTool.java
inventory_back/src/main/java/com/xixi/agent/service/ProcurementWarningAgentService.java
inventory_back/src/main/java/com/xixi/agent/service/impl/ProcurementWarningAgentServiceImpl.java
```

从第二个 Agent 开始，建议你统一成：

```
service 接口 + impl 实现类
```

这样和你原项目结构一致。

16.4 新建 WarningScanRequest

文件：

```
inventory_back/src/main/java/com/xixi/agent/dto/WarningScanRequest.java
```

代码：

```
package com.xixi.agent.dto;

import lombok.Data;

@Data
public class WarningScanRequest {
    private Integer days = 7;

    private String threadId;
}
```

16.5 新建 WarningItemVO

文件：

```
inventory_back/src/main/java/com/xixi/agent/vo/WarningItemVO.java
```

代码：

```
package com.xixi.agent.vo;

import lombok.Data;

@Data
public class WarningItemVO {
    private String riskLevel;

    private String bizType;

    private Long bizId;

    private String bizNo;

    private String problem;

    private String reason;

    private String suggestOwner;

    private String suggestAction;
}
```

16.6 新建 WarningScanVO

文件:

```
inventory_back/src/main/java/com/xixi/agent/vo/WarningScanVO.java
```

代码:

```
package com.xixi.agent.vo;

import lombok.Data;

import java.util.List;

@Data
public class WarningScanVO {
    private String summary;

    private List<WarningItemVO> items;

    private String aiSummary;
}
```

16.7 新建 5 个预警快照查询 VO

你可以偷懒，先统一用一个 VO。

文件:

```
inventory_back/src/main/java/com/xixi/agent/vo/WarningSnapshotVO.java
```

代码:

```
package com.xixi.agent.vo;

import lombok.Data;
import java.time.LocalDateTime;

@Data
public class WarningSnapshotVO {
    private Long bizId;

    private String bizNo;

    private String status;

    private Long supplierId;

    private String supplierName;

    private Long warehouseId;

    private String warehouseName;

    private LocalDateTime lastOperateTime;

    private Integer overdueDays;
}
```

16.8 新建 AgentWarningMapper

文件:

```
inventory_back/src/main/java/com/xixi/agent/mapper/AgentWarningMapper.java
```

代码:

```
package com.xixi.agent.mapper;

import com.xixi.agent.vo.WarningSnapshotVO;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;

import java.util.List;

@Mapper
public interface AgentWarningMapper {
    List<WarningSnapshotVO> getWaitConfirmOverdueOrders(@Param("days") Integer days);

    List<WarningSnapshotVO> getInProgressWithoutArrivalOrders(@Param("days") Integer days);

    List<WarningSnapshotVO> getPartialArrivalStuckOrders(@Param("days") Integer days);

    List<WarningSnapshotVO> getArrivedWithoutInboundRecords(@Param("days") Integer days);

    List<WarningSnapshotVO> getPendingInboundOverdueRecords(@Param("days") Integer days);
}
```

16.9 新建 AgentWarningMapper.xml

文件:

```
inventory_back/src/main/resources/com/xixi/mapper/AgentWarningMapper.xml
```

第一版先写简单一点，不追求 SQL 很炫。

代码:

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.xixi.agent.mapper.AgentWarningMapper">

    <select id="getWaitConfirmOverdueOrders"
resultType="com.xixi.agent.vo.WarningSnapshotVO">
        select po.id as bizId,
               po.order_no as bizNo,
               po.status as status,
               po.supplier_id as supplierId,
               s.name as supplierName,
               po.create_time as lastOperateTime,
               timestampdiff(day, po.create_time, now()) as overdueDays
        from purchase_order po
        left join supplier s on po.supplier_id = s.id
        where po.deleted = 0
              and po.status = 'WAIT_CONFIRM'
              and timestampdiff(day, po.create_time, now()) >= #{days}
    </select>

    <select id="getInProgressWithoutArrivalOrders"
resultType="com.xixi.agent.vo.WarningSnapshotVO">
        select po.id as bizId,
               po.order_no as bizNo,
               po.status as status,
               po.supplier_id as supplierId,
               s.name as supplierName,
               po.confirm_time as lastOperateTime,
               timestampdiff(day, po.confirm_time, now()) as overdueDays
        from purchase_order po
        left join supplier s on po.supplier_id = s.id
        where po.deleted = 0
              and po.status = 'IN_PROGRESS'
              and timestampdiff(day, po.confirm_time, now()) >= #{days}
              and not exists (
            select 1
            from arrival a
            where a.order_id = po.id
                  and a.deleted = 0
        )
    </select>

    <select id="getPartialArrivalStuckOrders"
resultType="com.xixi.agent.vo.WarningSnapshotVO">
        select po.id as bizId,
               po.order_no as bizNo,
               po.status as status,
               po.supplier_id as supplierId,
               s.name as supplierName,
               max(a.create_time) as lastOperateTime,
               timestampdiff(day, max(a.create_time), now()) as overdueDays

```

```

from purchase_order po
left join supplier s on po.supplier_id = s.id
left join arrival a on a.order_id = po.id and a.deleted = 0
where po.deleted = 0
      and po.status = 'PARTIAL_ARRIVAL'
group by po.id, po.order_no, po.status, po.supplier_id, s.name
having timestampdiff(day, max(a.create_time), now()) >= #{days}
</select>

<select id="getArrivedWithoutInboundRecords"
resultType="com.xixi.agent.vo.WarningSnapshotVO">
    select a.id as bizId,
           a.arrival_no as bizNo,
           a.status as status,
           a.warehouse_id as warehouseId,
           w.name as warehouseName,
           a.create_time as lastOperateTime,
           timestampdiff(day, a.create_time, now()) as overdueDays
    from arrival a
    left join warehouse w on a.warehouse_id = w.id
    where a.deleted = 0
          and timestampdiff(day, a.create_time, now()) >= #{days}
          and not exists (
                select 1
                from inbound i
                where i.arrival_id = a.id
                      and i.deleted = 0
            )
    </select>

<select id="getPendingInboundOverdueRecords"
resultType="com.xixi.agent.vo.WarningSnapshotVO">
    select i.id as bizId,
           i.inbound_no as bizNo,
           i.status as status,
           i.warehouse_id as warehouseId,
           w.name as warehouseName,
           i.create_time as lastOperateTime,
           timestampdiff(day, i.create_time, now()) as overdueDays
    from inbound i
    left join warehouse w on i.warehouse_id = w.id
    where i.deleted = 0
          and i.status = 'PENDING'
          and timestampdiff(day, i.create_time, now()) >= #{days}
    </select>
</mapper>

```

16.10 新建 ProcurementWarningSnapshotTool

这个 Tool 的思路和第一个 Agent 一样：先把结构化扫描结果给模型，而不是让模型自己猜。

文件：

```
inventory_back/src/main/java/com/xixi/agent/tool/ProcurementWarningSnapshotTool.java
```

代码:

```
package com.xixi.agent.tool;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.xixi.agent.dto.WarningScanRequest;
import com.xixi.agent.mapper.AgentWarningMapper;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.model.ToolContext;
import org.springframework.stereotype.Component;

import java.util.HashMap;
import java.util.Map;
import java.util.function.BiFunction;

@Component
@RequiredArgsConstructor
public class ProcurementWarningSnapshotTool implements BiFunction<WarningScanRequest, ToolContext, String> {
    private final AgentWarningMapper agentWarningMapper;
    private final ObjectMapper objectMapper;

    @Override
    public String apply(WarningScanRequest request, ToolContext toolContext) {
        try {
            Integer days = request == null || request.getDays() == null ? 7 : request.getDays();

            Map<String, Object> result = new HashMap<>();
            result.put("waitConfirmOverdue", agentWarningMapper.getWaitConfirmOverdueOrders(days));
            result.put("inProgressWithoutArrival", agentWarningMapper.getInProgressWithoutArrivalOrders(days));
            result.put("partialArrivalStuck", agentWarningMapper.getPartialArrivalStuckOrders(days));
            result.put("arrivedWithoutInbound", agentWarningMapper.getArrivedWithoutInboundRecords(days));
            result.put("pendingInboundOverdue", agentWarningMapper.getPendingInboundOverdueRecords(days));

            return objectMapper.writeValueAsString(Map.of(
                "success", true,
                "data", result
            ));
        } catch (Exception e) {
            return "{\"success\":false,\"message\":\"采购预警工具执行失败\"}";
        }
    }
}
```

16.11 新建 ProcurementWarningAgentService 接口

文件:

```
inventory_back/src/main/java/com/xixi/agent/service/ProcurementWarningAgentService.java
```

代码:

```
package com.xixi.agent.service;

import com.xixi.agent.dto.WarningScanRequest;
import com.xixi.agent.vo.WarningScanVO;

public interface ProcurementWarningAgentService {
    WarningScanVO scanWarnings(WarningScanRequest request);
}
```

16.12 新建 ProcurementWarningAgentServiceImpl

文件:

```
inventory_back/src/main/java/com/xixi/agent/service/impl/ProcurementWarningAgentServiceImpl.java
```

思路:

1. 先用 AgentWarningMapper 查 5 类风险
2. Java 组装成 List<WarningItemVO>
3. 再调用预警 Agent 生成 summary / aiSummary
4. 返回 WarningScanVO

代码:

```

package com.xixi.agent.service.impl;

import com.alibaba.cloud.ai.graph RunnableConfig;
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.xixi.agent.dto.WarningScanRequest;
import com.xixi.agent.mapper.AgentWarningMapper;
import com.xixi.agent.service.ProcurementWarningAgentService;
import com.xixi.agent.vo.WarningItemVO;
import com.xixi.agent.vo.WarningScanVO;
import com.xixi.agent.vo.WarningSnapshotVO;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
@RequiredArgsConstructor
public class ProcurementWarningAgentServiceImpl implements ProcurementWarningAgentService
{
    private final AgentWarningMapper agentWarningMapper;

    @Qualifier("procurementWarningAgent")
    private final ReactAgent procurementWarningAgent;

    @Override
    public WarningScanVO scanWarnings(WarningScanRequest request) {
        Integer days = request.getDays() == null ? 7 : request.getDays();

        List<WarningItemVO> items = new ArrayList<>();
        appendWarnings(items, agentWarningMapper.getWaitConfirmOverdueOrders(days),
            "HIGH", "PURCHASE_ORDER", "采购订单待供应商确认超时", "订单长时间停留在
WAIT_CONFIRM 状态", "SUPPLIER");
        appendWarnings(items, agentWarningMapper.getInProgressWithoutArrivalOrders(days),
            "HIGH", "PURCHASE_ORDER", "采购订单执行中但无到货", "订单进入执行中后长时间没有
到货记录", "SUPPLIER");
        appendWarnings(items, agentWarningMapper.getPartialArrivalStuckOrders(days),
            "MEDIUM", "PURCHASE_ORDER", "采购订单部分到货后停滞", "订单处于
PARTIAL_ARRIVAL 且长时间没有新到货", "SUPPLIER");
        appendWarnings(items, agentWarningMapper.getArrivedWithoutInboundRecords(days),
            "HIGH", "ARRIVAL", "到货后未生成入库单", "到货记录存在, 但仍未生成入库单",
"WAREHOUSE");
        appendWarnings(items, agentWarningMapper.getPendingInboundOverdueRecords(days),
            "MEDIUM", "INBOUND", "待确认入库单超时", "入库单长时间处于 PENDING 状态",
"WAREHOUSE");

        WarningScanVO vo = new WarningScanVO();
        vo.setItems(items);
        vo.setSummary("本次扫描共发现 " + items.size() + " 个执行风险。");
    }
}

```

```

        String threadId = request.getThreadId() == null ||
request.getThreadId().isBlank()
        ? "warning-scan-" + days
        : request.getThreadId();

        RunnableConfig config = RunnableConfig.builder()
            .threadId(threadId)
            .build();

        String prompt = ""
            我已经通过 Java 规则扫描出采购执行风险列表, 共 %s 条。
            请你基于工具 procurementWarningSnapshotTool 和下列风险列表, 生成一段简洁中文总
            结。

            需要包含:
            1. 高风险数量
            2. 中风险数量
            3. 最优先处理的风险类型
            4. 建议先由哪个角色处理
            """.formatted(items.size());

        try {
            AssistantMessage response = procurementWarningAgent.call(prompt, config);
            vo.setAiSummary(response.getText());
        } catch (Exception e) {
            vo.setAiSummary(null);
        }

        return vo;
    }

    private void appendWarnings(List<WarningItemVO> items,
                                List<WarningSnapshotVO> snapshots,
                                String riskLevel,
                                String bizType,
                                String problem,
                                String reason,
                                String owner) {
        if (snapshots == null || snapshots.isEmpty()) {
            return;
        }
        for (WarningSnapshotVO snapshot : snapshots) {
            WarningItemVO item = new WarningItemVO();
            item.setRiskLevel(riskLevel);
            item.setBizType(bizType);
            item.setBizId(snapshot.getBizId());
            item.setBizNo(snapshot.getBizNo());
            item.setProblem(problem);
            item.setReason(reason + ", 已超时 " + snapshot.getOverdueDays() + " 天");
            item.setSuggestOwner(owner);
            item.setSuggestAction("请优先处理 " + snapshot.getBizNo());
            items.add(item);
        }
    }

```

```
}
}
```

16.13 在 AgentConfig 中新增 procurementWarningAgent

在 `AgentConfig.java` 中追加：

```
@Bean
public ReactAgent procurementWarningAgent(ChatModel chatModel,
                                           ProcurementWarningSnapshotTool tool) {
    ToolCallback warningTool = FunctionToolCallback
        .builder("procurementWarningSnapshotTool", tool)
        .description("查询采购执行预警扫描结果，包括待确认超时、执行中无到货、部分到货停滞、已到  

        货未入库、待确认入库超时等风险")
        .inputType(WarningScanRequest.class)
        .build();

    ModelCallLimitHook limitHook = ModelCallLimitHook.builder()
        .runLimit(5)
        .exitBehavior(ModelCallLimitHook.ExitBehavior.ERROR)
        .build();

    return ReactAgent.builder()
        .name("procurement_warning_agent")
        .model(chatModel)
        .systemPrompt("""
            你是采购执行预警助手。
            你必须先调用工具 procurementWarningSnapshotTool 获取结构化预警数据，再生成
            总结。

            你不能编造风险，不允许新增不存在的订单、到货单、入库单。
            输出应聚焦：风险数量、风险等级、优先处理建议、建议处理角色。
            """)
        .tools(warningTool)
        .hooks(limitHook)
        .saver(new MemorySaver())
        .build();
}
```

16.14 在 AgentController 中补接口

新增：

```
private final ProcurementWarningAgentService procurementWarningAgentService;

@PostMapping("/warning/scan")
public Result scanWarnings(@RequestBody WarningScanRequest request) {
    return Result.success(procurementWarningAgentService.scanWarnings(request));
}
```

16.15 Apifox 测试

接口:

```
POST /agent/warning/scan
```

Body:

```
{
  "days": 3,
  "threadId": "warning-scan-001"
}
```

预期:

```
{
  "code": 200,
  "msg": "success",
  "data": {
    "summary": "本次扫描共发现 X 个执行风险。",
    "items": [
      {
        "riskLevel": "HIGH",
        "bizType": "PURCHASE_ORDER",
        "bizNo": "P02026040011",
        "problem": "采购订单执行中但无到货",
        "reason": "订单进入执行中后长时间没有到货记录, 已超时 3 天",
        "suggestOwner": "SUPPLIER",
        "suggestAction": "请优先处理 P02026040011"
      }
    ],
    "aiSummary": "..."
  }
}
```

16.16 这一节你应该重点理解什么

这一类 Agent 的本质不是“问答”，而是：

```
Java 规则先筛风险
LLM 只写总结
```

这比让模型自己扫描数据库稳定得多。

17. 第三个 Agent：供应商履约评分 Agent 详细落地步骤

供应商履约评分 Agent 最后做，因为它依赖的统计维度更多。

17.1 先理解这个 Agent 的职责

这个 Agent 不负责“审核供应商资料”，而是分析：

- 供应商确认是否及时
- 到货完成率如何
- 入库完成率如何
- 异常到货率高不高
- 是否适合继续合作

它是一个：

数据统计 + 规则打分 + LLM 评价

型 Agent。

17.2 推荐包结构

建议新增：

```
inventory_back/src/main/java/com/xixi/agent/dto/SupplierScoreRequest.java
inventory_back/src/main/java/com/xixi/agent/vo/SupplierScoreVO.java
inventory_back/src/main/java/com/xixi/agent/vo/SupplierPerformanceMetricsVO.java
inventory_back/src/main/java/com/xixi/agent/mapper/SupplierPerformanceMapper.java
inventory_back/src/main/resources/com/xixi/mapper/SupplierPerformanceMapper.xml
inventory_back/src/main/java/com/xixi/agent/tool/SupplierPerformanceSnapshotTool.java
inventory_back/src/main/java/com/xixi/agent/service/SupplierPerformanceAgentService.java
inventory_back/src/main/java/com/xixi/agent/service/impl/SupplierPerformanceAgentServiceImpl.java
```

17.3 新建 SupplierScoreRequest

文件：

```
inventory_back/src/main/java/com/xixi/agent/dto/SupplierScoreRequest.java
```

代码：

```
package com.xixi.agent.dto;

import lombok.Data;

@Data
public class SupplierScoreRequest {
    private Long supplierId;

    private Integer days = 30;

    private String threadId;
}
```

17.4 新建 SupplierPerformanceMetricsVO

文件:

```
inventory_back/src/main/java/com/xixi/agent/vo/SupplierPerformanceMetricsVO.java
```

代码:

```
package com.xixi.agent.vo;

import lombok.Data;

@Data
public class SupplierPerformanceMetricsVO {
    private Long supplierId;

    private String supplierName;

    private Integer totalOrderCount;

    private Integer completedOrderCount;

    private Integer cancelledOrderCount;

    private Integer abnormalArrivalCount;

    private Integer totalArrivalCount;

    private Double confirmRate;

    private Double arrivalCompletionRate;

    private Double inboundCompletionRate;

    private Double abnormalArrivalRate;
}
```

17.5 新建 SupplierScoreVO

文件:

```
inventory_back/src/main/java/com/xixi/agent/vo/SupplierScoreVO.java
```

代码:

```
package com.xixi.agent.vo;

import lombok.Data;

@Data
public class SupplierScoreVO {
    private Long supplierId;

    private String supplierName;

    private Integer score;

    private String level;

    private String confirmRate;

    private String arrivalCompletionRate;

    private String inboundCompletionRate;

    private String abnormalArrivalRate;

    private String analysis;

    private String suggestion;
}
```

17.6 新建 SupplierPerformanceMapper

文件:

```
inventory_back/src/main/java/com/xixi/agent/mapper/SupplierPerformanceMapper.java
```

代码:

```
package com.xixi.agent.mapper;

import com.xixi.agent.vo.SupplierPerformanceMetricsVO;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;

@Mapper
public interface SupplierPerformanceMapper {
    SupplierPerformanceMetricsVO getSupplierPerformanceMetrics(@Param("supplierId") Long
                                                                supplierId,
                                                                @Param("days") Integer
                                                                days);
}
```

17.7 新建 SupplierPerformanceMapper.xml

文件：

```
inventory_back/src/main/resources/com/xixi/mapper/SupplierPerformanceMapper.xml
```

第一版先写简单可跑通版：

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.xixi.agent.mapper.SupplierPerformanceMapper">
    <select id="getSupplierPerformanceMetrics"
resultType="com.xixi.agent.vo.SupplierPerformanceMetricsVO">
        select s.id as supplierId,
            s.name as supplierName,
            (
                select count(1)
                from purchase_order po
                where po.supplier_id = s.id
                    and po.deleted = 0
                    and po.create_time >= date_sub(now(), interval #{days} day)
            ) as totalOrderCount,
            (
                select count(1)
                from purchase_order po
                where po.supplier_id = s.id
                    and po.deleted = 0
                    and po.status = 'COMPLETED'
                    and po.create_time >= date_sub(now(), interval #{days} day)
            ) as completedOrderCount,
            (
                select count(1)
                from purchase_order po
                where po.supplier_id = s.id
                    and po.deleted = 0
                    and po.status in ('CANCELLED', 'CLOSED')
                    and po.create_time >= date_sub(now(), interval #{days} day)
            ) as cancelledOrderCount,
            (
                select count(1)
                from arrival a
                left join purchase_order po on a.order_id = po.id
                where po.supplier_id = s.id
                    and a.deleted = 0
                    and po.deleted = 0
                    and a.status = 'ABNORMAL'
                    and a.create_time >= date_sub(now(), interval #{days} day)
            ) as abnormalArrivalCount,
            (
                select count(1)
                from arrival a
                left join purchase_order po on a.order_id = po.id
                where po.supplier_id = s.id
                    and a.deleted = 0
                    and po.deleted = 0
                    and a.create_time >= date_sub(now(), interval #{days} day)
            ) as totalArrivalCount
        from supplier s
        where s.id = #{supplierId}
    
```

```
        and s.deleted = 0  
    </select>  
</mapper>
```

17.8 新建 SupplierPerformanceSnapshotTool

文件:

```
inventory_back/src/main/java/com/xixi/agent/tool/SupplierPerformanceSnapshotTool.java
```

代码:

```

package com.xixi.agent.tool;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.xixi.agent.dto.SupplierScoreRequest;
import com.xixi.agent.mapper.SupplierPerformanceMapper;
import com.xixi.agent.vo.SupplierPerformanceMetricsVO;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.model.ToolContext;
import org.springframework.stereotype.Component;

import java.util.Map;
import java.util.function.BiFunction;

@Component
@RequiredArgsConstructor
public class SupplierPerformanceSnapshotTool implements BiFunction<SupplierScoreRequest,
ToolContext, String> {
    private final SupplierPerformanceMapper supplierPerformanceMapper;
    private final ObjectMapper objectMapper;

    @Override
    public String apply(SupplierScoreRequest request, ToolContext toolContext) {
        try {
            if (request == null || request.getSupplierId() == null) {
                return objectMapper.writeValueAsString(Map.of(
                    "success", false,
                    "message", "supplierId不能为空"
                ));
            }

            Integer days = request.getDays() == null ? 30 : request.getDays();
            SupplierPerformanceMetricsVO metrics =

supplierPerformanceMapper.getSupplierPerformanceMetrics(request.getSupplierId(), days);

            if (metrics == null) {
                return objectMapper.writeValueAsString(Map.of(
                    "success", false,
                    "message", "供应商不存在"
                ));
            }

            return objectMapper.writeValueAsString(Map.of(
                "success", true,
                "data", metrics
            ));
        } catch (Exception e) {
            return "{\n\"success\":false,\n\"message\":\n\"供应商履约工具执行失败\n\"}";
        }
    }
}

```

17.9 新建 SupplierPerformanceAgentService 接口

文件:

```
inventory_back/src/main/java/com/xixi/agent/service/SupplierPerformanceAgentService.java
```

代码:

```
package com.xixi.agent.service;

import com.xixi.agent.dto.SupplierScoreRequest;
import com.xixi.agent.vo.SupplierScoreVO;

public interface SupplierPerformanceAgentService {
    SupplierScoreVO scoreSupplier(SupplierScoreRequest request);
}
```

17.10 新建 SupplierPerformanceAgentServiceImpl

文件:

```
inventory_back/src/main/java/com/xixi/agent/service/impl/SupplierPerformanceAgentServiceI
mpl.java
```

核心思路:

1. 查供应商指标
2. Java 先算分
3. Java 先定等级
4. Agent 再生成分析与建议

代码:

```

package com.xixi.agent.service.impl;

import com.alibaba.cloud.ai.graph RunnableConfig;
import com.alibaba.cloud.ai.graph.agent.ReactAgent;
import com.xixi.agent.dto.SupplierScoreRequest;
import com.xixi.agent.mapper.SupplierPerformanceMapper;
import com.xixi.agent.service.SupplierPerformanceAgentService;
import com.xixi.agent.vo.SupplierPerformanceMetricsVO;
import com.xixi.agent.vo.SupplierScoreVO;
import lombok.RequiredArgsConstructor;
import org.springframework.ai.chat.messages.AssistantMessage;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class SupplierPerformanceAgentServiceImpl implements
SupplierPerformanceAgentService {
    private final SupplierPerformanceMapper supplierPerformanceMapper;

    @Qualifier("supplierPerformanceAgent")
    private final ReactAgent supplierPerformanceAgent;

    @Override
    public SupplierScoreVO scoreSupplier(SupplierScoreRequest request) {
        if (request.getSupplierId() == null) {
            SupplierScoreVO vo = new SupplierScoreVO();
            vo.setAnalysis("supplierId不能为空");
            vo.setSuggestion("请传入供应商ID。");
            return vo;
        }

        Integer days = request.getDays() == null ? 30 : request.getDays();
        SupplierPerformanceMetricsVO metrics =

supplierPerformanceMapper.getSupplierPerformanceMetrics(request.getSupplierId(), days);

        if (metrics == null) {
            SupplierScoreVO vo = new SupplierScoreVO();
            vo.setSupplierId(request.getSupplierId());
            vo.setAnalysis("供应商不存在");
            vo.setSuggestion("请确认 supplierId 是否正确。");
            return vo;
        }

        SupplierScoreVO vo = new SupplierScoreVO();
        vo.setSupplierId(metrics.getSupplierId());
        vo.setSupplierName(metrics.getSupplierName());

        if (metrics.getTotalOrderCount() == null || metrics.getTotalOrderCount() == 0) {
            vo.setScore(0);
            vo.setLevel("数据不足");
        }
    }
}

```

```

        vo.setConfirmRate("0.00%");
        vo.setArrivalCompletionRate("0.00%");
        vo.setInboundCompletionRate("0.00%");
        vo.setAbnormalArrivalRate("0.00%");
        vo.setAnalysis("该供应商存在，但当前统计周期内暂无采购订单履约数据，暂无法形成有效履约评
价。");

        vo.setSuggestion("建议扩大统计周期后重新分析，例如查看最近90天或180天。");
        return vo;
    }

    double confirmRate = safeRate(metrics.getCompletedOrderCount(),
metrics.getTotalOrderCount());
    double arrivalCompletionRate = safeRate(metrics.getCompletedOrderCount(),
metrics.getTotalOrderCount());
    double inboundCompletionRate = safeRate(metrics.getCompletedOrderCount(),
metrics.getTotalOrderCount());
    double abnormalArrivalRate = safeRate(metrics.getAbnormalArrivalCount(),
metrics.getTotalArrivalCount());

    int confirmScore = (int) Math.round(confirmRate * 20);
    int arrivalScore = (int) Math.round(arrivalCompletionRate * 30);
    int inboundScore = (int) Math.round(inboundCompletionRate * 20);
    int abnormalScore = (int) Math.round((1 - abnormalArrivalRate) * 20);
    int cancelScore = (int) Math.round((1 -
safeRate(metrics.getCancelledOrderCount(), metrics.getTotalOrderCount())) * 10);

    int totalScore = confirmScore + arrivalScore + inboundScore + abnormalScore +
cancelScore;
    totalScore = Math.max(0, Math.min(totalScore, 100));

    vo.setScore(totalScore);
    vo.setLevel(resolveLevel(totalScore));
    vo.setConfirmRate(formatRate(confirmRate));
    vo.setArrivalCompletionRate(formatRate(arrivalCompletionRate));
    vo.setInboundCompletionRate(formatRate(inboundCompletionRate));
    vo.setAbnormalArrivalRate(formatRate(abnormalArrivalRate));

    String threadId = request.getThreadId() == null ||
request.getThreadId().isBlank()
        ? "supplier-score-" + request.getSupplierId()
        : request.getThreadId();

    RunnableConfig config = RunnableConfig.builder()
        .threadId(threadId)
        .build();

    String prompt = ""
        请分析供应商 %s 在最近 %s 天的履约表现。
        注意：该供应商已由后端 Java 查询确认存在，supplierId=%s，supplierName=%s。
        如果各项指标为 0，只能说明当前统计周期内履约数据不足，不能说供应商不存在。

        Java 规则已算出：
        总分：%s

```

等级: %s
 确认及时率: %s
 到货完成率: %s
 入库完成率: %s
 异常到货率: %s

请你调用工具 `supplierPerformanceSnapshotTool` 核对数据, 并生成:

1. 总体评价
2. 主要优点
3. 主要风险
4. 合作建议

严禁输出“供应商不存在”, 除非工具明确返回 `success=false` 且 `message=供应商不存在`。

```
"".formatted(
    metrics.getSupplierName(),
    days,
    metrics.getSupplierId(),
    metrics.getSupplierName(),
    totalScore,
    vo.getLevel(),
    vo.getConfirmRate(),
    vo.getArrivalCompletionRate(),
    vo.getInboundCompletionRate(),
    vo.getAbnormalArrivalRate()
);

try {
    AssistantMessage response = supplierPerformanceAgent.call(prompt, config);
    vo.setAnalysis(response.getText());
    vo.setSuggestion("建议结合 AI 分析结果决定后续合作策略。");
} catch (Exception e) {
    vo.setAnalysis("已完成供应商履约评分, 但 AI 分析摘要生成失败。");
    vo.setSuggestion("可先依据分数和指标进行人工判断。");
}

return vo;
}

private double safeRate(Integer numerator, Integer denominator) {
    if (denominator == null || denominator == 0) {
        return 0D;
    }
    int n = numerator == null ? 0 : numerator;
    return (double) n / denominator;
}

private String formatRate(double rate) {
    return String.format("%.2f%%", rate * 100);
}

private String resolveLevel(int score) {
    if (score >= 90) {
        return "优秀";
    }
}
```

```

    }
    if (score >= 75) {
        return "良好";
    }
    if (score >= 60) {
        return "一般";
    }
    return "较差";
}
}

```

17.11 在 AgentConfig 中新增 supplierPerformanceAgent

追加:

```

@Bean
public ReactAgent supplierPerformanceAgent(ChatModel chatModel,
                                           SupplierPerformanceSnapshotTool tool) {
    ToolCallback supplierTool = FunctionToolCallback
        .builder("supplierPerformanceSnapshotTool", tool)
        .description("根据供应商ID和统计周期查询供应商履约指标，包括订单数量、完成数量、取消数量、异常到货数量等")
        .inputType(SupplierScoreRequest.class)
        .build();

    ModelCallLimitHook limitHook = ModelCallLimitHook.builder()
        .runLimit(5)
        .exitBehavior(ModelCallLimitHook.ExitBehavior.ERROR)
        .build();

    return ReactAgent.builder()
        .name("supplier_performance_agent")
        .model(chatModel)
        .systemPrompt("""
            你是供应商履约评分助手。
            你必须先调用工具 supplierPerformanceSnapshotTool 获取供应商履约指标，再生成
            分析。

            你不能编造订单数量、异常数量、评分结果。
            输出需要聚焦：总体评价、优势、风险、合作建议。
            """)
        .tools(supplierTool)
        .hooks(limitHook)
        .saver(new MemorySaver())
        .build();
}

```

17.12 在 AgentController 中补接口

新增:

```
private final SupplierPerformanceAgentService supplierPerformanceAgentService;

@PostMapping("/supplier/score")
public Result scoreSupplier(@RequestBody SupplierScoreRequest request) {
    return Result.success(supplierPerformanceAgentService.scoreSupplier(request));
}
```

17.13 Apifox 测试

接口:

POST /agent/supplier/score

Body:

```
{
  "supplierId": 1,
  "days": 30,
  "threadId": "supplier-score-001"
}
```

预期:

```
{
  "code": 200,
  "msg": "success",
  "data": {
    "supplierId": 1,
    "supplierName": "供应商001",
    "score": 86,
    "level": "良好",
    "confirmRate": "90.00%",
    "arrivalCompletionRate": "88.00%",
    "inboundCompletionRate": "85.00%",
    "abnormalArrivalRate": "6.00%",
    "analysis": "...",
    "suggestion": "..."
  }
}
```

17.14 这一节你应该重点理解什么

供应商评分 Agent 的本质不是“让模型打分”，而是：

Java 规则算分
模型负责解释分数

这是业务 Agent 非常关键的思路：

- 事实用 SQL 查
- 分数用 Java 算
- 说明用模型写

还要注意一个边界：

供应商存在 != 当前统计周期内有履约数据

如果 `totalOrderCount = 0`，应该由 Java 直接返回“数据不足”，不要把这种情况交给模型分析。否则模型可能把“没有订单数据”误说成“供应商不存在”。

17A. 从第二个 Agent 开始推荐统一成 service + impl

你现在第一个 Agent 为了最小跑通，`ProcessDiagnosisAgentService` 直接写成了具体类，这没问题。

但从第二个 Agent 开始，我建议统一成：

service 接口
service/impl 实现类

原因：

1. 和你原项目结构一致
2. 三个 Agent 会越来越像标准业务模块
3. 方便后面做测试、替换实现、扩展更多 Agent
4. Controller 依赖接口更清晰

18. 常见问题处理

18.1 依赖版本缺失

报错：

```
'dependencies.dependency.version' is missing
```

原因：

dependency 没写 version，也没配置 BOM

解决：

要么加 BOM
要么给 dependency 写 version

18.2 找不到 ChatModel Bean

报错类似：

```
Parameter 0 of method inventoryChatAgent required a bean of type
'org.springframework.ai.chat.model.ChatModel'
```

可能原因：

- DashScope starter 没引入成功
- API Key 配置错误
- 自动配置没生效

临时解决方式：手动创建 ChatModel。

可以参考官方 Quick Start：

```
DashScopeApi dashScopeApi = DashScopeApi.builder()
    .apiKey(System.getenv("AI_DASHSCOPE_API_KEY"))
    .build();

ChatModel chatModel = DashScopeChatModel.builder()
    .dashScopeApi(dashScopeApi)
    .build();
```

需要的导包通常是：

```
import com.alibaba.cloud.ai.dashscope.api.DashScopeApi;
import com.alibaba.cloud.ai.dashscope.chat.DashScopeChatModel;
import org.springframework.ai.chat.model.ChatModel;
```

18.3 Tool 没被调用

可能原因：

- Tool 名称不清楚
- Tool 描述太模糊
- Prompt 没要求必须先调用工具

解决：

1. Tool 名称写清楚：getPurchaseOrderSnapshot
2. description 写清楚
3. systemPrompt 里写：必须先调用工具

18.4 Agent 无限循环

解决：

```
ModelCallLimitHook limitHook = ModelCallLimitHook.builder()
    .runLimit(5)
    .exitBehavior(ModelCallLimitHook.ExitBehavior.ERROR)
    .build();
```

并在 Agent 里：

```
.hooks(limitHook)
```

18.5 返回内容不是 JSON

第一版不要强制 Agent 输出 JSON。

建议：

```
结构化字段由 Java 返回
aiSummary 只是文本
```

也就是：

```
{
  "currentStage": "入库确认阶段",
  "blockReason": "已到货但未入库",
  "aiSummary": "模型生成的一段解释"
}
```

不要一开始要求模型完整生成结构化 JSON。

18.6 /agent 接口 401

原因：

```
Spring Security 拦截了
```

临时测试放行：

```
.requestMatchers(
    "/auth/login",
    "/error",
    "/agent/**"
).permitAll()
```

正式版本不要全部放行，应按角色控制。

19. 第一版完成标准

满足下面这些，就算第一版 Agent 跑通：

1. 项目能正常启动
2. /agent/chat 能返回模型内容
3. /agent/diagnose/order 能接收 orderNo
4. 后端能查出订单执行快照
5. Java 能判断 currentStage / blockReason / suggestAction
6. Agent 能生成 aiSummary
7. Apifox 能完整跑通

第一版不要求：

1. 采购预警 Agent 完成
2. 供应商评分 Agent 完成
3. 持久化会话记忆
4. Graph Workflow
5. RAG
6. 多 Agent 协作

20. 最终推荐开发顺序

严格按这个来：

1. 修 pom.xml 依赖
2. 配 DashScope API Key
3. 写 /agent/chat
4. 写 AgentQueryMapper
5. 写 PurchaseOrderSnapshotTool
6. 写 processDiagnosisAgent Bean
7. 写 OrderDiagnosisRule
8. 写 ProcessDiagnosisAgentService
9. 写 /agent/diagnose/order
10. Apifox 测 P02026040011
11. 再做 /agent/warning/scan
12. 最后做 /agent/supplier/score

21. 简历表述

第一版跑通后可以写：

基于 Spring AI Alibaba Agent Framework 构建采购流程阻塞诊断 Agent，将采购订单、到货、入库等业务数据封装为只读 Tool，结合业务状态机规则和 LLM 总结能力，实现订单执行阶段识别、阻塞原因分析和下一步处理建议生成。

三个 Agent 都完成后可以写：

基于 Spring AI Alibaba Agent Framework 构建采购执行智能体体系，将采购订单、到货、入库、库存、供应商履约等领域 Service 封装为 Agent Tools，结合业务状态机规则、采购执行预警规则和供应商评分模型，实现订单阻塞诊断、采购执行预警和供应商履约评分。

22. 结论

你现在不要一上来做大而全 Agent。

最稳路线是：

先做流程阻塞诊断 Agent

因为它：

- 输入最简单，只需要订单号。
- 最容易验证 Tool Calling。
- 复用你现有采购、到货、入库链路。
- 后续可以复用到采购执行预警和供应商履约评分。

第一个 Agent 跑通后，后面的两个 Agent 只是扩大查询范围和增加评分规则。